SCSM2223 — GAP-CLOSER DRILL SHEET

Every concept found in the course materials that Sets A, B and C do not directly test

HIGH items in red — drill these first. Result of a file-by-file audit of all slides, labs and solution code.

1. DOM & BOM — missed items (Ch3 labs)

HIGH **classList — add / remove / toggle (used in BOTH Ch3 labs, never tested)**

→ el.classList.add("note-item") · el.classList.remove("dark") · el.classList.toggle("is-hidden"). Two-argument form: el.classList.toggle("is-hidden", condition) — adds the class when condition is true, removes it when false. The Kanban lab explicitly requires filtering cards this way, NOT via style.display.

HIGH **The three BOM dialogs and their return values**

→ alert(msg) → returns undefined (just informs). confirm(msg) → true/false (tested). prompt(msg, default) → the typed STRING, or null on Cancel — always check for null before using.

HIGH **BOM window & navigator (a full pillar of the Ch3 lab)**

```
function updateWindowSize() {
  document.getElementById("size").textContent =
    window.innerWidth + " x " + window.innerHeight;
}
window.addEventListener("resize", updateWindowSize);
console.log(navigator.userAgent);   // browser identification string
```

HIGH **Delete an item from a state array by id (recurs in two labs)**

```
const index = notes.findIndex(n => n.id === id);
if (index !== -1) {
  notes.splice(index, 1);   // remove 1 element at that position
}
// alternative already drilled: notes = notes.filter(n => n.id !== id)
```

2. Async, fetch & jQuery — missed items (Ch4–5)

HIGH **fetch PUT and DELETE (Sets tested GET + POST only — this completes CRUD)**

```
await fetch(url + "/3", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(updatedBook)
});

await fetch(url + "/3", { method: "DELETE" });   // no body needed
```

HIGH **Authorization header in a fetch call (Ch4 material bridges into JWT)**

```
fetch(url, {
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer " + token
  }
});
```

MED **Constructing a Promise (authoring, not just consuming)**

```
function loadScript(src) {
  return new Promise(function (resolve, reject) {
    const s = document.createElement("script");
    s.src = src;
    s.onload = () => resolve(s);
    s.onerror = () => reject(new Error("Load failed: " + src));
    document.head.append(s);
  });
}
```

MED **XMLHttpRequest actual syntax (concept was tested; code was not)**

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "data.json");
xhr.onload = function () {
  if (xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
  }
};
xhr.onerror = function () { console.log("Network error"); };
xhr.send();
// older idiom: onreadystatechange + readyState === 4 (done)
```

MED **jQuery beyond $.get/.text() — one-line meanings**

→ .val() read/write input value · .hide()/.show()/.toggle() visibility · .addClass()/.removeClass()/.toggleClass() · .append()/.remove()/.empty() DOM edit · $.getJSON(url, cb) · $.ajax({url, method, success, error}) full form · chain .done().fail().always() ≈ then/catch/finally.

MED **AbortController — cancel/timeout a fetch**

```
const controller = new AbortController();
setTimeout(() => controller.abort(), 10000);   // 10 s timeout
fetch(url, { signal: controller.signal });
```

MED **WebSockets (last link of the taught XHR→AJAX→fetch→async evolution)**

→ Full-duplex, bidirectional communication over ONE long-lived TCP connection — server can push without a request. Use cases from the slide: chat apps, online gaming, live data feeds.

MED **Debounce (named lab requirement, 500 ms)**

→ Delay running a handler until the user has STOPPED typing for X ms — clear the previous timer on each keystroke (clearTimeout + setTimeout). Purpose: avoid firing a search request on every keypress.

3. Vue — missed items (Ch6–8, Ch13)

HIGH **onMounted — THE standard initial-data-load hook (used in every lab, never tested)**

```
import { ref, onMounted } from 'vue'
const students = ref([]); const loading = ref(false); const error = ref('')

async function load() {
  loading.value = true
  try { students.value = (await getStudents()).data }
  catch (e) { error.value = 'Could not load data' }
  finally { loading.value = false }
}
onMounted(load)   // runs once when the component is inserted
```

HIGH **useRouter (write) vs useRoute (read) — programmatic navigation**

```
import { useRouter, useRoute } from 'vue-router'
const router = useRouter()          // actions
router.push({ name: 'todo', params: { id: 7 } })
router.back()
const route = useRoute()            // current info
const id = route.params.id          // ← always a STRING
```

HIGH **route.params gotcha — params are STRINGS**

→ `find(t => t.id === route.params.id)` fails when ids are numbers (`7 === "7"` is false). Fix: `Number(route.params.id)` before strict comparison. Classic bug-spot question.

HIGH **:class / :style object binding (used in every lab component)**

→ `<p :class="{ active: isActive, done: t.done }">` — class applied when value truthy. `<p :style="{ color: c, fontSize: size + 'px' }">`.

HIGH **Router guard + requiresAuth (Ch13 lab self-check item)**

```
{ path: '/profile', component: ProfileView, meta: { requiresAuth: true } }

router.beforeEach((to) => {
  const auth = useAuthStore()
  if (to.meta.requiresAuth && !auth.token) {
    return { path: '/login', query: { redirect: to.fullPath } }
  }
})
```

HIGH **Pinia auth store with localStorage persistence (repeated 3× in Ch13)**

```
export const useAuthStore = defineStore('auth', () => {
  const token = ref(localStorage.getItem('token') || '')
  const isAuthenticated = computed(() => token.value !== '')
  function setToken(t) {
    token.value = t
    localStorage.setItem('token', t)
  }
  function logout() { token.value = ''; localStorage.removeItem('token') }
  return { token, isAuthenticated, setToken, logout }
})
```

HIGH **Axios interceptors — the CODE (concept tested, code repeated 3× in materials)**

```
api.interceptors.request.use((cfg) => {
```

```
  const token = useAuthStore().token
  if (token) cfg.headers.Authorization = `Bearer ${token}`
  return cfg
})
api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) useAuthStore().logout()
    return Promise.reject(err)
  }
)
```

MED **Event modifiers beyond .prevent**

→ .stop → stopPropagation() (stops bubbling — contrast with .prevent!) · .once → handler fires a single time · @keyup.enter / @keyup.esc key modifiers.

MED **Project folder semantics (recall-tier)**

→ views/ route-level pages · components/ reusable pieces · router/index.js routes · stores/ Pinia · App.vue root shell · main.js entry (createApp/use/mount) · @/ is the Vite alias for src/.

MED **Small Vue facts**

→ <template v-if="..."> groups elements without adding a wrapper div · catch-all route { path: '/:pathMatch(.*)*' } renders the 404 view · router-link gets the automatic class router-link-active · interpolation is HTML-escaped and takes ONE expression (no if/for statements).

MED **JSON Server query conventions (slides teach them; Slim is still the exam backend)**

→ GET /students?faculty=FSKSM filter · GET /students?_sort=gpa&_order=desc sort · GET /students?q=aisyah full-text search.

4. Backend (Slim / PDO / PHP) — missed items (Ch9–11)

HIGH **password_hash — the REGISTRATION side (Sets only tested password_verify)**

```
$hash = password_hash($password, PASSWORD_DEFAULT);   // store $hash, never the password
// bonus taught in slides:
if (password_needs_rehash($row['password_hash'], PASSWORD_DEFAULT)) { /* re-hash & save */ }
```

HIGH **Duplicate registration → 409 (Ch11 lab pattern)**

```
if ($this->users->emailExists($body['email'])) {
    return $this->json($res, ['error' => 'Email already registered'], 409);
}
```

HIGH **THREE route-callable syntaxes (Slim accepts all — both appear across chapters)**

→ 1) closure `function (Request $q, Response $s) {…}` 2) string `BookController::class . ':create'` 3) array `[BookController::class, 'index']`.

MED **phpdotenv — how .env actually reaches PHP**

```
// composer require vlucas/phpdotenv
Dotenv\Dotenv::createImmutable(__DIR__ . '/..')->safeLoad();
$dbName = $_ENV['DB_NAME'];
```

MED **Generating the JWT secret (appears 3× across chapters — lecturer favourite)**

→ `bin2hex(random_bytes(32))` → 64-char hex secret for `JWT_SECRET` in .env; rotating it invalidates ALL sessions.

MED **ORDER BY whitelist — the SECOND SQL-injection angle**

→ Placeholders protect VALUES, not identifiers — you cannot bind a column name. Sortable columns must be whitelisted: `$allowed = ['name','matricNo','gpa']; if (!in_array($sort, $allowed)) $sort = 'name';` “Never let the client choose an arbitrary column name.”

MED **PDO SQLSTATE quick table (matching-question fodder)**

→ `2002` can’t connect (MySQL not running) · `28000` access denied (credentials) · `42S02` table not found · `HY093` invalid parameter number (same named placeholder used twice) · “could not find driver” = pdo_mysql not enabled.

MED **Misc backend facts**

→ 401 responses in the lab also send `WWW-Authenticate: Bearer` · MySQL DECIMAL comes back as a STRING → `Number(s.gpa).toFixed(2)` · PHP is stateless between requests — that is WHY Ch9 persisted to books.json and why Ch10 (DB) exists · dynamic UPDATE: build SET clause only from fields present (`array_key_exists`).

5. Security — missed items (Ch12)

HIGH **DOM-based XSS — the THIRD type (Sets tested stored + reflected only)**

→ Happens entirely on the client: JavaScript reads attacker-controlled input like `location.hash` and writes it to `innerHTML` without sanitising. Defense: never put user input into innerHTML — use `textContent`.

HIGH **Security headers table (dedicated slide + lab middleware — memorize all six)**

```
X-Content-Type-Options: nosniff        → browser respects declared MIME type
X-Frame-Options: DENY                  → stops clickjacking (no iframes)
Referrer-Policy: no-referrer-when-downgrade
Strict-Transport-Security: max-age=63072000; includeSubDomains   → HTTPS-only
Content-Security-Policy: default-src 'self'   → controls script/style sources
Permissions-Policy: camera=(), microphone=()  → disables unused sensors
```

HIGH **CSRF — own slide, contrasted with IDOR**

→ CSRF rides credentials the browser sends AUTOMATICALLY (cookies): a malicious site triggers a request to your API and the cookie goes along. Defenses: `SameSite` cookies, CSRF tokens, re-authentication. KEY LINE: Bearer-token APIs are largely NOT vulnerable — the token is attached manually by your JS, not auto-sent. CSRF = forged REQUEST; IDOR = missing OWNERSHIP check.

HIGH **Where to store the JWT — the comparison table**

→ `localStorage`: survives refresh but readable by any injected script (XSS risk). `HttpOnly + Secure + SameSite cookie`: invisible to JS (XSS-safe) but auto-sent → needs CSRF protection. In-memory: safest, lost on refresh. The course app uses localStorage via the Pinia store — know the trade-off.

MED **Refresh tokens (slide content, out of lab scope)**

→ Short-lived ACCESS token (e.g. 1 h) + long-lived REFRESH token; each refresh issues a new pair (rotation); revocation via a denylist keyed by the `jti` claim.

MED **OWASP Top 10 codes as shown in lecture (8 items)**

→ A01 Broken Access Control · A02 Cryptographic Failures · A03 Injection · A04 Insecure Design · A05 Security Misconfiguration · A07 Authentication Failures · A08 Software/Data Integrity · A09 Logging & Monitoring Failures. (Slide omits A06 Vulnerable Components and A10 SSRF.)

MED **Middleware ORDER trap — two rules that sound alike**

→ Error middleware: added LAST → runs FIRST (outermost) → catches everything. SecurityHeaders middleware: added FIRST → runs LAST on the way OUT → stamps headers on every response. Same onion, opposite intent — a perfect trick question.

MED **Security odds and ends**

→ Exclude `OPTIONS` preflight from auth middleware or CORS dies with 401 · XSS-safe JSON output flags: `JSON_HEX_TAG|JSON_HEX_AMP|JSON_HEX_APOS|JSON_HEX_QUOT` · `filter_var` also validates URL (`FILTER_VALIDATE_URL`) and IP · logging DO: auth events, 403s, rate-limit hits; DON'T: plaintext passwords, full JWTs, national IDs · reusable `Validator` class: required()/intRange()/email() then validate() → errors array.

6. Deployment & Capacitor — missed items (Ch13)

HIGH **capacitor.config.json — the exact fields**

```
{
  "appId": "com.utm.books",
  "appName": "UTM Books",
  "webDir": "dist",
  "server": { "androidScheme": "https" }
}
```

MED **cap init full syntax + signing**

→ npx cap init books-app com.utm.books --web-dir=dist · signing key: keytool -genkey -v -keystore release.keystore -alias books -keyalg RSA -keysize 2048 -validity 10000 · key values live in gradle.properties · Play Console listing costs US$25 · internal/closed testing before production.

MED **Operations tail**

→ A /api/health endpoint for smoke tests · full smoke test = register → login → CRUD flow before announcing · monitoring (UptimeRobot), error tracking (Sentry), backups + rollback plan.

7. Material discrepancies — answer strategy if they appear

⚠ DELETE status: Ch9 slide best-practice says 204 No Content; the lab code returns 200 with a JSON body. Concept question → say 204 (or "200 with body, per our lab convention") and state your assumption. Both defensible if justified.

⚠ PUT vs PATCH: theory (tested) says PUT replaces the WHOLE resource; the lab's PUT actually accepts partial bodies and no PATCH route is ever coded. Conceptual question → answer THEORY. Lab-code question → follow the lab.

⚠ OWASP list in the lecture slide has 8 of 10 items. If asked "per the lecture", give the 8; if asked for the official Top 10, A06 = Vulnerable & Outdated Components, A10 = SSRF.

⚠ Ch8 teaches JSON Server (slides) AND Express+MySQL (lab) — but the Dr's tips name the exam backend as PHP/Slim + PDO. Express syntax is LOW risk; the transferable ideas (prepared statements, pools, 3-tier, ER_DUP_ENTRY→409) are already covered via Slim/PDO. Know JSON Server's query strings as recall only.

⚠ Two Slim callable styles appear in different chapters (string 'Class:method' in Ch9 slides, array [Class::class,'method'] in labs) — both are correct; don't "correct" one to the other in a spot-the-error question.

— END OF DRILL SHEET — sit the three sets first; read this sheet twice; you now hold the entire examinable surface. —